

SIMATS ENGINEERING
CO2_ASSESSMENT TOOL3_ Dry Run Evaluation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192365045
Name	Shaik Mahaboob Basha

COURSE OUTCOME COVERED

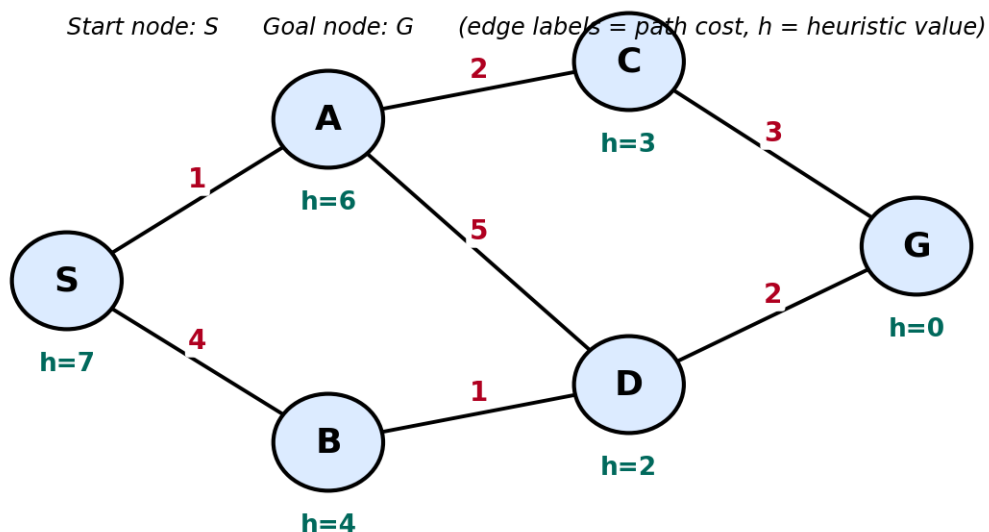
CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

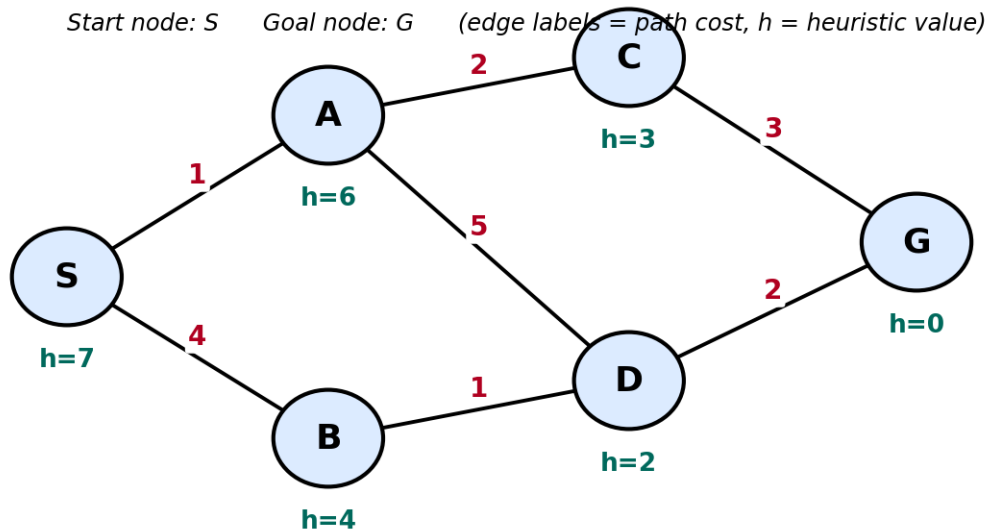
Total Marks:50

Q1. Consider a suitable state-space graph (with heuristic values $h(n)$ assigned to each node) of your choice for a given problem. Perform a dry run of Greedy Best-First Search from the start node to the goal node. Clearly show the frontier/open list, the node selected for expansion at each step, and the final path obtained.



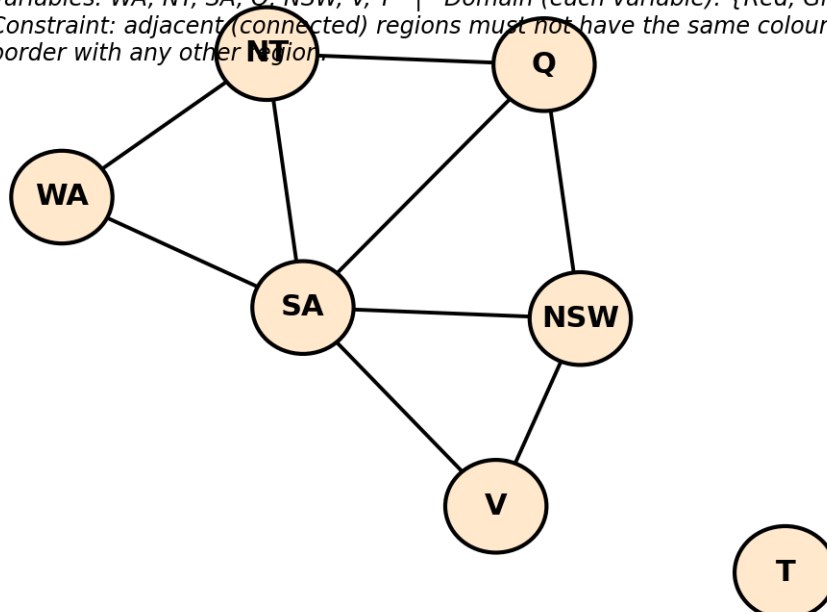
Q2. For a suitable weighted state-space graph (with edge costs and heuristic values $h(n)$) of your choice, perform a dry run of A* Search from the start node to the goal

node. Show the $g(n)$, $h(n)$ and $f(n)$ values computed at every step, the order in which nodes are expanded, and the final optimal path along with its total cost.



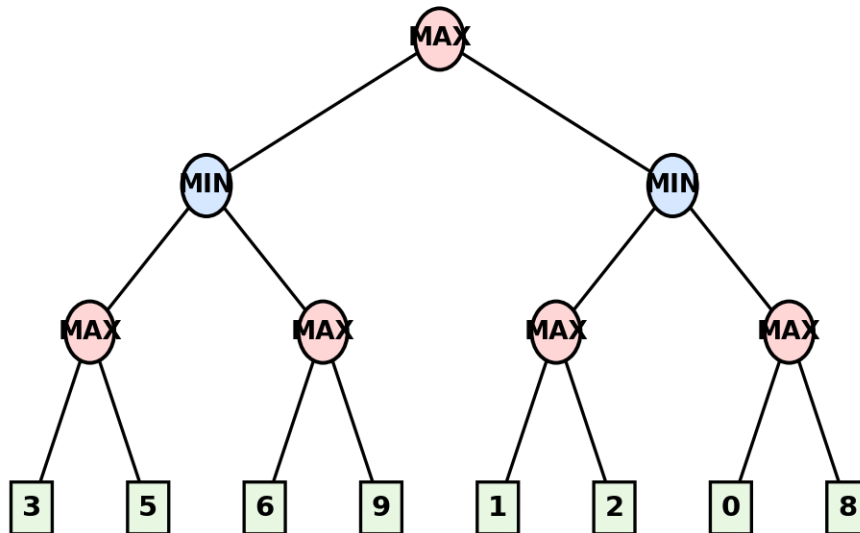
Q3. Formulate a given Constraint Satisfaction Problem (e.g., Map Colouring / N-Queens) by clearly stating the variables, their domains, and the constraints involved. Perform a dry run of the Backtracking Search algorithm, showing each variable assignment, every constraint check, and any backtracking steps, until a complete solution (or failure) is reached.

Variables: WA, NT, SA, Q, NSW, V, T | Domain (each variable): {Red, Green, Blue}
 Constraint: adjacent (connected) regions must not have the same colour. T has no shared border with any other region.



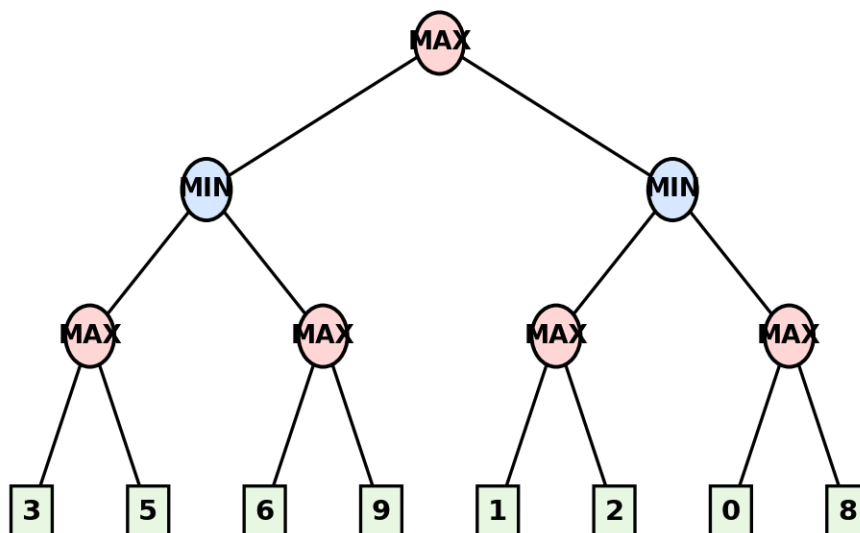
Q4. For a given two-player game tree of your choice (at least 3 levels deep), perform a dry run of the Minimax algorithm. Show the utility values at the leaf/terminal nodes and the backed-up values computed at every internal node, and state the best move for the MAX player with justification.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Q5. Using the same (or a different) game tree from Q4, perform a dry run of the Minimax algorithm with Alpha-Beta Pruning. Show the α and β values maintained at each node during the traversal, clearly indicate which branches get pruned and why, and state the final best move obtained.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.

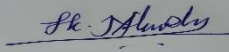


Code of Conduct:

I Shaik Mahaboob Basha / 192365045 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Dry Run Evaluation

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Problem Formulation & Setup (states/nodes, heuristic values or CSP variables, domains and constraints correctly identified)	Accurately identifies all required elements (states, heuristics/costs, or variables/domains/constraints) with clear, correct notation.	Identifies most required elements correctly, with only minor omissions or notational errors that do not affect the dry run.	Identifies some required elements; noticeable errors or missing components affect the later steps.	Little or no correct problem formulation; required elements are largely missing or incorrect.	10
Correct Application of Algorithm Procedure (expansion order, backtracking, or pruning as per the standard method)	Follows the exact algorithmic procedure at every step with no deviation from the standard method.	Follows the procedure correctly for most steps, with only one or two minor deviations that do not change the outcome.	Several steps deviate from the correct procedure, though the overall approach is recognisable.	Algorithm steps are largely incorrect, or the procedure is not followed.	10
Accuracy of Computations (g(n), h(n), f(n) values, minimax backed-up values, or α - β updates)	All computed values are accurate and consistent throughout the dry run.	Minor calculation errors are present but do not significantly affect the	Multiple calculation errors are present that affect the correctness of	Calculations are mostly incorrect or missing.	10

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
		final outcome.	intermediate steps.		
Correctness of Final Outcome (final path, CSP solution, or best move obtained)	Final result is completely correct and matches the expected optimal outcome.	Final result is mostly correct, with only a minor deviation from the optimal outcome.	Final result is only partially correct.	Final result is incorrect or not obtained.	10
Clarity of Presentation & Justification (trace tables/trees/diagrams, explanation of each step)	Dry run is presented clearly using well-organised tables/trees/diagrams, with every step explained and justified.	Dry run is reasonably clear and mostly organised, with minor gaps in explanation.	Dry run presentation is unclear or poorly organised, with limited justification.	Dry run is disorganised or illegible, with little to no justification.	10
				Total	50

Answers for above questions

1. Dry Run of Greedy Best-First Search

Start Node: S

Goal Node: G

Step 1: Heuristic Values

Node Heuristic h(n)

S 7

A 6

B 4

C 3

D 2

G 0

Greedy Best-First Search Evaluation Function:

$f(n)=h(n)$

The algorithm always expands the node with the smallest heuristic value.

Dry Run

Initial State

Open List (Frontier):

[S(7)]

Closed List:

[]

Selected Node:

S

Step 1: Expand S

Children of S:

A (h=6)

B (h=4)

Open List:

[B(4), A(6)]

Closed List:

[S]

Selected Node:

B

Step 2: Expand B

Child of B:

D (h=2)

Open List:

[D(2), A(6)]

Closed List:

[S, B]

Selected Node:

D

Step 3: Expand D

Children of D:

G (h=0)

A (already in Open List)

Open List:

[G(0), A(6)]

Closed List:

[S, B, D]

Selected Node:

G

Step 4: Expand G

The goal node is reached.

Search Stops.

Order of Node Expansion

$S \rightarrow B \rightarrow D \rightarrow G$

Final Path Obtained

$S \rightarrow B \rightarrow D \rightarrow G$

Path Cost

Using the edge costs:

$S \rightarrow B = 4$

$B \rightarrow D = 1$

$D \rightarrow G = 2$

Total Cost

$4+1+2=7$

2. Dry Run of A* Search

Start Node: S

Goal Node: G

A* Evaluation Function

$$f(n)=g(n)+h(n)$$

Where:

$g(n)$ = Cost from the start node to node n

$h(n)$ = Heuristic estimate to the goal

$f(n)$ = Total estimated cost

Heuristic Values

Node $h(n)$

S 7

A 6

B 4

C 3

D 2

G 0

Step 1: Start at S

$$g(S)=0, h(S)=7, f(S)=0+7=7$$

Open List

Node	g	h	f
S	0	7	7

Expand: S

Step 2: Expand S

Successors:

A

$$g=0+1=1$$

$$h=6$$

$$f=1+6=7$$

B

$$g=0+4=4$$

$$h=4$$

$$f=4+4=8$$

Open List

Node	g	h	f
A	1	6	7
B	4	4	8

Expand: A (lowest f)

Step 3: Expand A

Successors:

C

$$g=1+2=3$$

$$h=3$$

$$f=3+3=6$$

D

$$g=1+5=6$$

$$h=2$$

$$f=6+2=8$$

Open List

Node	g	h	f
C	3	3	6
B	4	4	8

D 6 2 8

Expand: C

Step 4: Expand C

Successor:

G

$$g=3+3=6$$

$$h=0$$

$$f=6+0=6$$

Open List

Node	g	h	f
G	6	0	6
B	4	4	8
D	6	2	8

Expand: G

Since G is the goal node, the search stops.

Order of Node Expansion

$S \rightarrow A \rightarrow C \rightarrow G$

Final Optimal Path

$S \rightarrow A \rightarrow C \rightarrow G$

Total Path Cost

Using edge costs:

$$S \rightarrow A = 1$$

$$A \rightarrow C = 2$$

$$C \rightarrow G = 3$$

$$\text{Total Cost} = 1 + 2 + 3 = 6$$

3. Constraint Satisfaction Problem (Map Colouring) – Backtracking Search Dry Run

Step 1: Formulate the CSP

Variables

The variables represent the Australian states and territories:

WA (Western Australia)

NT (Northern Territory)

SA (South Australia)

Q (Queensland)

NSW (New South Wales)

V (Victoria)

T (Tasmania)

Domains

Each variable can take one of the following colours:

{Red, Green, Blue}

Constraints

Adjacent regions must not have the same colour.

The adjacency constraints are:

$$WA \neq NT$$

$$WA \neq SA$$

$$NT \neq SA$$

$$NT \neq Q$$

$$SA \neq Q$$

SA != NSW

SA != V

Q != NSW

NSW != V

Step 2: Dry Run of Backtracking Search

Variable Order

WA → NT → SA → Q → NSW → V → T

Step 1: Assign WA

Try Red

No constraint violated.

Assignment

WA = Red

Step 2: Assign NT

Try Red

Constraint violated

WA = NT = Red

Backtrack and try another colour.

Try Green

Valid

Assignment

WA = Red

NT = Green

Step 3: Assign SA

Try Red

Violates

WA = SA = Red

Try Green

Violates

NT = SA = Green

Try Blue

Valid

Assignment

WA = Red

NT = Green

SA = Blue

Step 4: Assign Q

Try Red

Valid

Q is adjacent to NT and SA only.

NT = Green

SA = Blue

No conflict.

Assignment

WA = Red

NT = Green

SA = Blue

Q = Red

Step 5: Assign NSW

Try Red

Conflict

Q = NSW = Red

Try Green

Valid

Adjacent states:

SA = Blue

Q = Red

No conflict.

Assignment

WA = Red

NT = Green

SA = Blue

Q = Red

NSW = Green

Step 6: Assign V

Try Red

Valid

Adjacent states:

SA = Blue

NSW = Green

No conflict.

Assignment

WA = Red

NT = Green

SA = Blue

Q = Red

NSW = Green

V = Red

Step 7: Assign T

Tasmania has no neighboring states.

Try Red

Valid

Assignment

WA = Red

NT = Green

SA = Blue

Q = Red

NSW = Green

V = Red

T = Red

Final Solution

Variable	Colour
----------	--------

WA	Red
----	-----

NT	Green
----	-------

SA	Blue
----	------

Q	Red
---	-----

NSW	Green
-----	-------

V	Red
---	-----

T	Red
---	-----

Constraint Checking

WA != NT

WA != SA
NT != SA
NT != Q
SA != Q
SA != NSW
SA != V
Q != NSW
NSW != V
All constraints are satisfied.
Backtracking Performed
Only two backtracking steps were required:
NT: Red → Green
SA: Red → Green → Blue

4. Dry Run of the Minimax Algorithm

Step 1: Game Tree

Root Node: MAX
Level 1: MIN
Level 2: MAX
Level 3: Terminal (Leaf) Nodes
The terminal node utility values are:
Left Subtree:
(3, 5)
(6, 9)
Right Subtree:
(1, 2)
(0, 8)

Step 2: Evaluate Leaf Nodes

The utility values are already given.

3 5 6 9 1 2 0 8

Step 3: Evaluate MAX Nodes (Bottom-Up)

MAX Node 1

Children:

3, 5

MAX chooses the larger value.

$\max(3, 5) = 5$

Value backed up:

MAX = 5

MAX Node 2

Children:

6, 9

$\max(6, 9) = 9$

Value backed up:

MAX = 9

MAX Node 3

Children:

1, 2

$\max(1, 2) = 2$

Value backed up:

MAX = 2

MAX Node 4

Children:

0, 8

$\max(0,8)=8$

Value backed up:

MAX = 8

Step 4: Evaluate MIN Nodes

Left MIN Node

Children:

5, 9

MIN chooses the smaller value.

$\min(5,9)=5$

Value backed up:

MIN = 5

Right MIN Node

Children:

2, 8

$\min(2,8)=2$

Value backed up:

MIN = 2

Step 5: Evaluate Root MAX Node

Children:

5, 2

MAX chooses the larger value.

$\max(5,2)=5$

Therefore,

Root Value = 5

Compute MAX values:

$\text{MAX}(3,5) = 5$

$\text{MAX}(6,9) = 9$

$\text{MAX}(1,2) = 2$

$\text{MAX}(0,8) = 8$

Compute MIN values:

$\text{MIN}(5,9) = 5$

$\text{MIN}(2,8) = 2$

Compute root MAX:

$\text{MAX}(5,2) = 5$

Best Move for MAX Player

The root MAX player compares the two available moves:

Left subtree $\rightarrow$ Value = 5

Right subtree $\rightarrow$ Value = 2

Since $5 > 2$, the MAX player selects the left subtree.

5. Dry Run of Minimax with X-Y Pruning

Initial Values

Root (MAX): $X = -\text{Infinity}$, $Y = +\text{Infinity}$

The tree is evaluated from left to right.

Step 1: Evaluate Left MIN Node

First MAX Node

Leaf values: 3, 5

$$\max(3,5) = 5$$

MIN updates:

$$X = -\text{Infinity}$$

$$Y = \min(+\text{Infinity}, 5) = 5$$

Second MAX Node

Receives:

$$X = -\text{Infinity}$$

$$Y = 5$$

Evaluate first leaf:

$$\text{Value} = 6$$

MAX updates:

$$X = \max(-\text{Infinity}, 6) = 6$$

Since

$$X \geq Y$$

$$6 \geq 5$$

Y-cutoff occurs.

The second leaf 9 is pruned because it cannot affect the MIN node's final decision.

Left MIN returns:

$$\min(5,6)=5$$

Step 2: Update Root MAX

Current value at Root:

$$X = \max(-\text{Infinity}, 5) = 5$$

$$Y = +\text{Infinity}$$

Step 3: Evaluate Right MIN Node

Receives:

$$X = 5$$

$$Y = +\text{Infinity}$$

Third MAX Node

Leaf values:

1

2

MAX chooses:

$$\max(1,2)=2$$

MIN updates:

$$Y = \min(+\text{Infinity}, 2) = 2$$

Now

$$Y \leq X$$

$$2 \leq 5$$

X-cutoff occurs.

Therefore, the entire fourth MAX subtree (0, 8) is pruned because it cannot improve the Root MAX value.

Right MIN returns 2.

Step 4: Evaluate Root MAX

Root compares:

$$\text{Left MIN} = 5$$

$$\text{Right MIN} = 2$$

$\max(5, 2) = 5$

(X) and (Y) Values

Step	Node	X	Y	Action
1	Root MAX	$-\text{Infinity}$	$+\text{Infinity}$	Start
2	Left MIN	$-\text{Infinity}$	5	After first MAX
3	Second MAX	6	5	Y-cutoff, prune 9
4	Root MAX	5	$+\text{Infinity}$	Update X
5	Right MIN	5	2	X-cutoff
6	Fourth MAX	Pruned	Pruned	Entire subtree skipped

Pruned Branches

Leaf node 9 was pruned because:

$X = 6 \geq Y = 5$

Entire subtree (0, 8) was pruned because:

$Y = 2 \leq X = 5$

These branches cannot change the final Minimax decision.

Final Best Move

The Root MAX player compares:

Left subtree = 5

Right subtree = 2

Since

$5 > 2$

the MAX player chooses the left subtree.